

Mini-Project for AI Programming MED6 – 2026

Clara Marie Pedersen – Study no.20233918

Demo video: <https://youtu.be/EfWtL8fDM4w>

GitHub: <https://github.com/Otterly-Cute/AIProgrammingMiniProject>

All code used in this project as well as figures and animations can be seen in above GitHub repository.

Acknowledgements:

Artificial intelligence tools were used during the development of this project. OpenAI's ChatGPT was used to assist with code development, debugging, and refactoring, as well as generating suggestions for data visualisation and analysis. ChatGPT was also used to help shorten, revise, and improve the clarity of written sections of the report.

All generated code, text, and suggestions were reviewed, tested, and adapted by the author before inclusion in the final submission. Responsibility for the final implementation, analysis, and conclusions remains with the author.

1. Introduction

Efficient route planning is a critical issue in warehouse and retail settings, where workers must collect items from multiple locations while minimizing travel distance. As the number of items increases, finding the optimal picking sequence becomes increasingly challenging due to the large number of possible route combinations.

This project investigates the use of artificial intelligence approaches to optimize warehouse picking routes. A warehouse environment is represented as a two-dimensional grid containing obstacles, item placements, and a starting position. The shortest paths between locations are calculated using the A* pathfinding algorithm and stored in a distance matrix.

To determine the most efficient collection order, a Genetic Algorithm (GA) is used. The algorithm evolves candidate routes through selection, crossover, and mutation operations, with the objective of minimizing total travel distance. Different parent selection methods and hyperparameter configurations are evaluated to determine their impact on solution quality and computational performance.

The performance of the proposed approach is assessed by comparing optimized routes against random routes, analysing the effect of selection strategies and hyperparameters, and evaluating scalability as the number of items increases.

2. Warehouse Setup

The warehouse environment was represented as a two-dimensional grid consisting of walkable cells and obstacles. Obstacles represent warehouse shelves that cannot be traversed, while the remaining cells represent accessible floor space. A fixed starting location was defined to represent the picker's initial position.

Ten item locations were distributed throughout the warehouse and served as collection targets. To investigate scalability ten additional item location where used. Experiments were conducted using subsets containing 5, 10, 15, and 20 items. Each item was assigned a unique identifier (A–T) to simplify route visualisation and analysis.

Figure 1 illustrates the warehouse layout used throughout the experiments, including shelf locations, starting position, and item locations. The environment provides a realistic navigation problem in which routes must be planned around obstacles while visiting all required items.

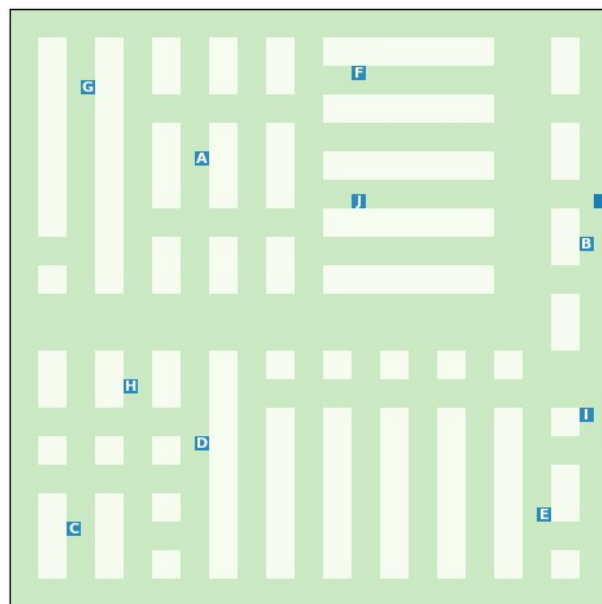


Figure 1, Warehouse Layout with Items and Start

The objective of the optimization process is to determine the order in which items should be collected so that the total travel distance is minimized while ensuring that every item is visited exactly once before returning to the starting position.

3. Route Optimization

3.1 A* Pathfinding and Distance Matrix

To determine travel distances within the warehouse, the A* pathfinding algorithm was used. A* is a graph search algorithm that finds the shortest path between two locations by combining the actual distance travelled with a heuristic estimate of the remaining distance to the target.

For each pair of locations, including the start position and all item locations, A* was executed to calculate the shortest traversable path while avoiding shelves. The resulting distances were stored in a distance matrix, where each matrix entry represents the shortest travel distance between two locations.

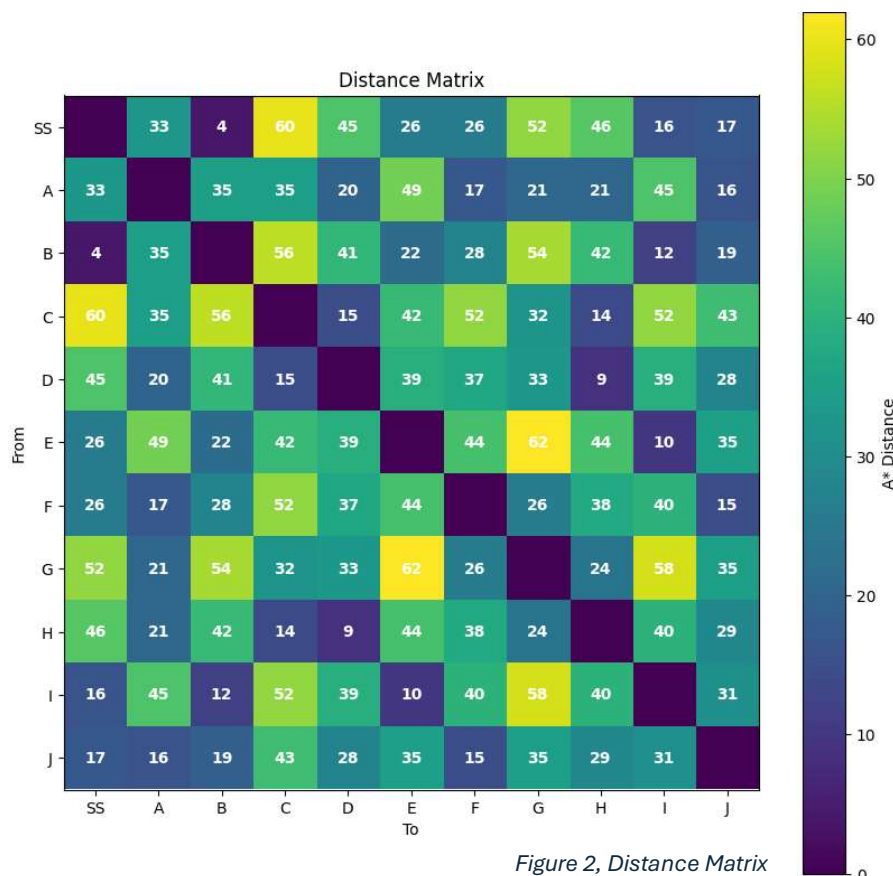


Figure 2, Distance Matrix

Precomputing these distances significantly reduces computational cost during optimization, as the Genetic Algorithm can evaluate routes using simple matrix lookups instead of repeatedly executing A* searches.

Figure 2 shows the generated distance matrix. Higher values indicate locations that are further apart within the warehouse, while lower values indicate locations that can be reached more quickly.

3.2 Genetic Algorithm

A Genetic Algorithm (GA) was used to optimize the order in which items are collected. The problem is closely related to the Travelling Salesman Problem (TSP), where the objective is to find the shortest route that visits a set of locations exactly once.

Each candidate solution, known as a chromosome, represents a complete picking route. The chromosome is encoded as a permutation of item indices, where the sequence determines the order in which items are collected.

The fitness of a route is determined by its total travel distance. This distance consists of the path from the starting position to the first item, the distances between consecutive items, and the distance required to return to the start position after the final item has been collected. Routes with shorter distances are considered better solutions.

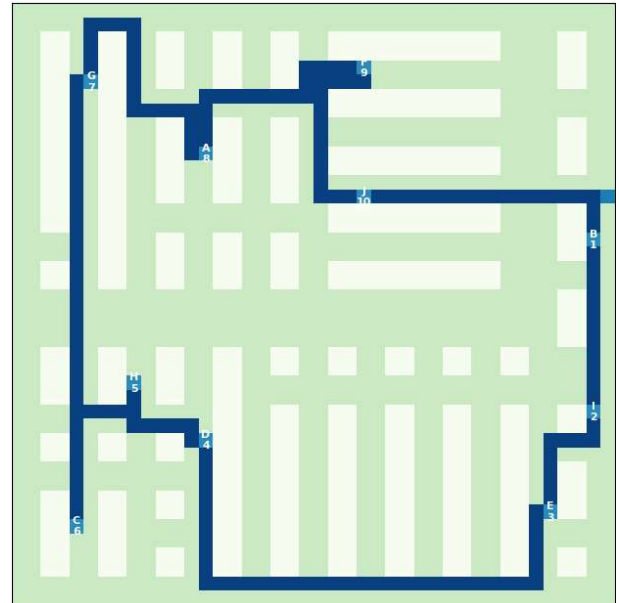


Figure 3, Optimized Route

The algorithm begins by generating a random population of routes. During each generation, parent routes are selected, combined using crossover, and modified through mutation to produce new candidate solutions. Over successive generations, the population evolves towards shorter and more efficient routes.

4. Evaluation

4.1 Random Route vs Optimized Route

To assess the effectiveness of the Genetic Algorithm, optimized routes were compared with randomly generated routes. A random route visits all items exactly once but does not apply any optimization strategy.

The optimized routes consistently produced shorter travel distances than random routes. This demonstrates that the Genetic Algorithm successfully identifies more efficient item collection sequences and reduces unnecessary movement within the warehouse.

Figure 4 shows an example of a random route, which can be compared with the optimized route generated by the Genetic Algorithm as seen in Figure 3. The optimized solution follows a more structured path through the warehouse and avoids excessive backtracking.

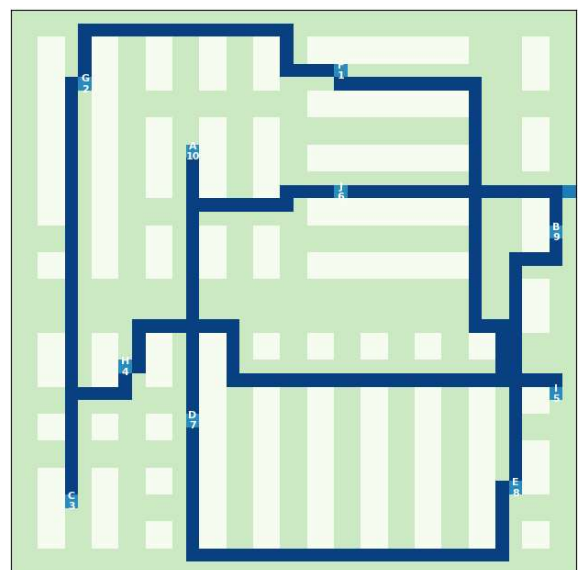


Figure 4, Random Route

4.2 Parent Selection Analysis

Several parent selection methods were evaluated, including Random Selection, Tournament Selection, Roulette Wheel Selection, Rank Selection, and Stochastic Universal Sampling (SUS).

The results indicate that fitness-based selection methods generally outperform random selection, producing shorter routes and more consistent convergence. Tournament Selection achieved the best overall performance, generating the shortest average route distances across multiple runs.

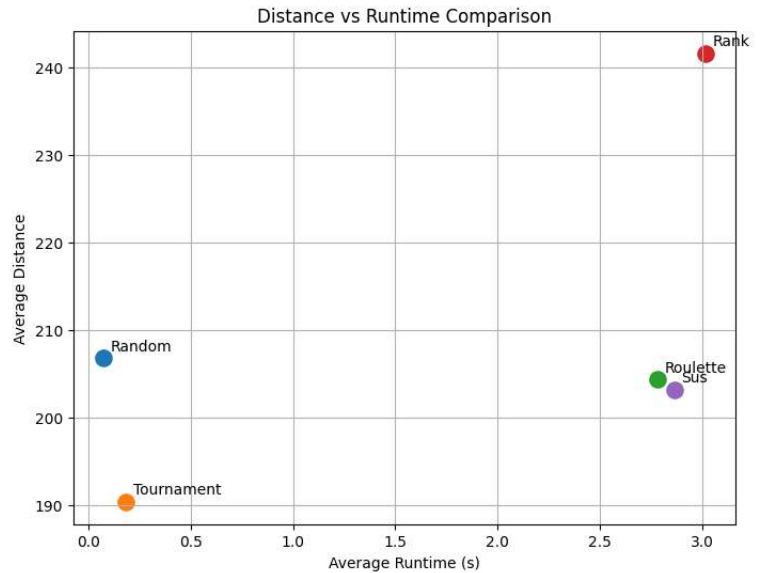


Figure 5, Distance vs Runtime Comparison

Figure 5 shows the comparison between the average route distance and the average runtime obtained by each selection method.

4.3 Hyperparameter Analysis

The influence of population size, mutation rate, and generation count was investigated through a series of experiments.

Increasing the population size generally improved solution quality by providing greater diversity within the search process, although larger populations also increased runtime. Similarly, increasing the number of generations allowed the algorithm to continue improving solutions, but with diminishing returns after a certain point.

Mutation rate was found to have a significant impact on performance. Very low mutation rates reduced exploration of the search space, while excessively high rates disrupted promising solutions. Moderate mutation rates produced the best balance between exploration and exploitation.

A parameter grid search was performed to identify effective combinations of population size, mutation rate, and generation count. Figure 6 presents a heatmap of the resulting performance measurements for population size x generations. The other two heatmaps can be found in *main.ipynb*.

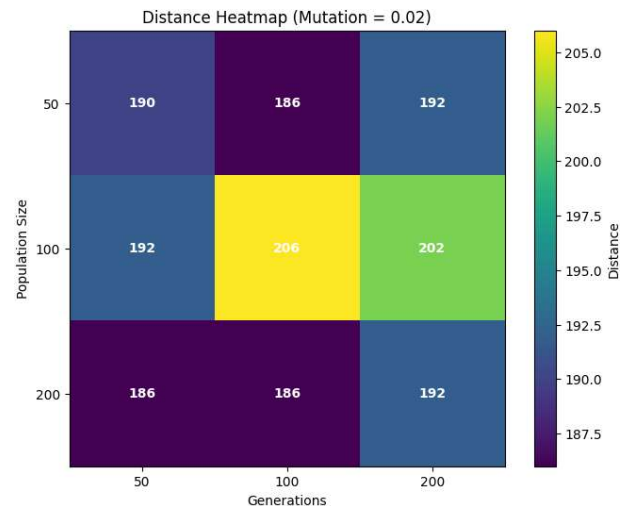


Figure 6, Distance Heatmap

5. Scalability Analysis

To evaluate how the proposed approach performs as problem size increases, experiments were conducted using item sets containing 5, 10, 15, and 20 items. For each configuration, the Genetic Algorithm was executed using the same parameter settings and the resulting route distance and runtime were recorded.

The results show that runtime increases as the number of items grows. This behaviour is expected because larger item sets produce a significantly larger search space and require a larger distance matrix. In addition, route evaluation becomes more computationally expensive as chromosomes contain more locations. Although runtime increased with problem size, the Genetic Algorithm was still able to generate high-quality routes for all tested item counts. This demonstrates that the approach remains effective as warehouse complexity increases.

Figure 7 shows the relationship between item count and runtime, as well as the relationship between item count and corresponding route distances produced by the algorithm.

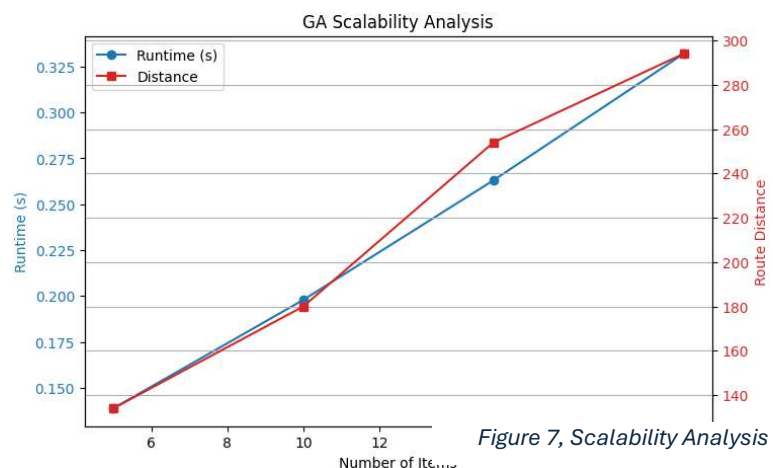


Figure 7, Scalability Analysis

6. Future Improvements and Conclusion

The results demonstrate that combining A* pathfinding with a Genetic Algorithm provides an effective approach for warehouse route optimization. By using A* to calculate realistic travel distances and a Genetic Algorithm to determine item collection order, the system consistently produced shorter routes than random solutions.

Among the tested selection methods, fitness-based approaches generally outperformed random selection, while hyperparameter tuning further improved solution quality. The scalability experiments showed that the method remains effective as the number of items increases, although computational cost grows with problem size.

Several improvements could be explored in future work. More advanced crossover and mutation operators could be investigated, and adaptive parameter tuning could be used to improve convergence. Additional warehouse constraints, such as multiple pickers, item priorities, or dynamic obstacles, could also be incorporated to create a more realistic optimization problem.

Overall, the project demonstrates how Artificial Intelligence techniques can be applied to solve practical route optimization problems and improve efficiency in warehouse environments.

Resources:

Genetic Algorithms – Fundamentals

https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_fundamentals.htm

Genetic Algorithms - Parent Selection

https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm

Genetic Algorithms - Crossover

https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_crossover.htm

Crossover in Genetic Algorithm

<https://www.geeksforgeeks.org/machine-learning/crossover-in-genetic-algorithm/>

A* algorithm and its Heuristic Search Strategy in Artificial Intelligence

https://www.geeksforgeeks.org/artificial-intelligence/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/?utm_source=chatgpt.com

ML – Convergence of Genetic Algorithms

<https://www.geeksforgeeks.org/machine-learning/ml-convergence-of-genetic-algorithms/>